ID	Cas de test	Entrées / Procédure	Résultat attendu
T1	**Conformité au contrat CSV**	Appeler le client pour Reims (id_commune="51454"), horizon PROJ_2041_2050, modèles MRI_AGCM3_2_S et CMCC_CM2_VHR4.	Génère un fichier CSV en UTF-8 (séparateur ;, décimales avec .) avec les 8 colonnes attendues et 730 lignes de données.
T2	**Préservation du zéro initial**	Appeler le client avec un département de 01 à 09 : id_commune="05023".	La colonne id_commune du CSV conserve bien la chaîne "05023" et ne la transforme pas en 5023.
T3	**Gestion des valeurs manquantes**	Simuler un JSON de l'API où temperature_2m_max vaut null.	La ligne CSV produite contient une case vide (;;) pour cette température, et non un zéro.
T4	**Extraction de elevation**	Exécuter l'appel nominal sur Reims (51454).	Le client génère un fichier JSON séparé contenant l'altitude renvoyée par Open-Meteo pour vérifier Dim_Commune.
T5	**Fonctionnement du cache**	Exécuter T1 (cache vide) en mesurant le temps, puis relancer T1 immédiatement.	Le premier appel crée un fichier dans data/cache/. Le second est instantané car aucune requête réseau n'est effectuée.

ID	Cas de test	Entrées / Procédure	Résultat attendu
T6	**Validation des paramètres**	Passer une date de fin antérieure à la date de début, ou des coordonnées hors bornes.	Le code lève une exception ValueError côté Python avant de consommer le moindre quota API.
T7	**Gestion des pannes (HTTP 429)**	Simuler le code HTTP 429 (quota dépassé) ou un timeout de 0.001s.	Le client effectue 3 réessais avec temporisation avant de remonter une erreur propre qui ne crashe pas l'application.